

A Second-Order Optimization-Based Adaptive Attack Method for Deep Convolutional Neural Networks

Jin Ding¹, Jie-Chao Zhao, Yong-Zhi Sun, Ping Tan¹, *Member, IEEE*, Ji-En Ma¹, *Member, IEEE*, and You-Tong Fang¹

Abstract—Deep convolutional neural networks (DCNNs) are vulnerable to small perturbations. Currently, first-order optimization-based adversarial attack methods are the mainstream methods for generating perturbations, such as PGD. It is known that, the second-order optimization method can converge faster than the first-order methods. Therefore, it is natural to expect the performance of second-order optimization-based attack methods should be better than that of first-order methods. However, the existing second-order attack methods do not demonstrate superior performance compared to first-order attack methods. The reason behind is that, these second-order attack methods employ the same sign function as first-order methods to generate pixel perturbations. Hence, in this article, we propose a second-order optimization-based adaptive attack method, and at its core, an adaptive perturbation generation strategy is designed. We evaluate the proposed method on multiple datasets and various models. The experimental results demonstrate that the proposed second-order optimization-based adaptive attack method is able to generate adversarial examples that cause misclassification of DCNNs in fewer iterations than existing second-order attack methods and PGD, and has competitive time complexity.

Index Terms—Adaptive perturbation generation, adversarial attacks, deep neural networks (DCNNs), robustness, second-order optimization.

I. INTRODUCTION

DEEP learning, a focal point of research in artificial intelligence in recent years, has made notable achievements in

Received 6 April 2025; accepted 14 August 2025. Date of publication 4 September 2025; date of current version 3 December 2025. This work was supported in part by the Key R&D Plan Projects in Zhejiang Province under Grant 2023C01243, in part by the National Natural Science Foundation of China under Grant 52293424, and in part by the Open Foundation of the State Key Laboratory of Fluid Power and Mechatronic Systems under Grant GZKF-202329. Associate Editor: Y. Sui. (*Corresponding authors: Jin Ding; Ping Tan.*)

Jin Ding is with the School of Automation and Electrical Engineering and Key Institute of Robotics of Zhejiang Province, Zhejiang University of Science and Technology, Hangzhou 310023, China, and also with the State Key Laboratory of Fluid Power and Mechatronic Systems, Zhejiang University, Hangzhou 310027, China (e-mail: jding@zust.edu.cn).

Jie-Chao Zhao, Yong-Zhi Sun, and Ping Tan are with the School of Automation and Electrical Engineering and Key Institute of Robotics of Zhejiang Province, Zhejiang University of Science and Technology, Hangzhou 310023, China (e-mail: zjiechao@126.com; sunyongzhi@hotmail.com; 115011@zust.edu.cn).

Ji-En Ma and You-Tong Fang are with the School of Electrical Engineering, Zhejiang University, Hangzhou 310027, China (e-mail: majien@zju.edu.cn; youtong@zju.edu.cn).

Digital Object Identifier 10.1109/TR.2025.3600691

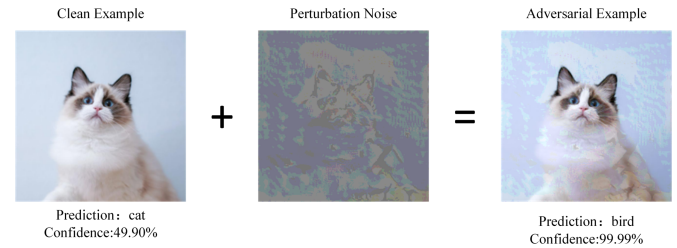


Fig. 1. Clean example with small noise can lead to misclassification by convolutional neural networks.

areas, such as autonomous driving [1], intelligent security [2], and smart healthcare [3]. However, these high-performance deep models, especially deep convolutional neural networks (DCNNs), often exhibit vulnerabilities under adversarial attacks. Szegedy [4] and others discovered that small input perturbations, almost imperceptible to the human eyes, can lead to misclassifications by the DCNNs. This phenomenon is not only present in image recognition [5] but also occurs in other tasks, such as voice recognition [6] and natural language processing [7]. Fig. 1 illustrates how a “cat” image being slightly perturbed, can result in a prediction of “bird” with high confidence.

With respect to the adversarial attack methods, the researchers have proposed a number of effective and efficient approaches, e.g., fast gradient sign method (FGSM) [8] and projected gradient descent (PGD) [9]. It is noted that, most existing attack methods are based on first-order optimization, which needs multiple iterations to achieve the desired attack outcome [10]. When targeting the DCNNs with intricate defense mechanisms, these first-order optimization-based adversarial attack methods frequently fall short of expectations [11].

It is known that, the second-order optimization methods, which use Hessian matrix to better determine the descent directions and step sizes, can converge faster than first-order optimization methods [12]. Recently, with the progress in the Hessian matrix approximation computation, the second-order optimization methods begun to apply in practical engineering tasks [13], e.g., training of deep neural networks [14]. It is natural to expect the second-order optimization-based attack methods are more efficient and effective than first-order attack methods. However, existing second-order attack methods [41], [42], [43] do not demonstrate superior performance compared to first-order attack

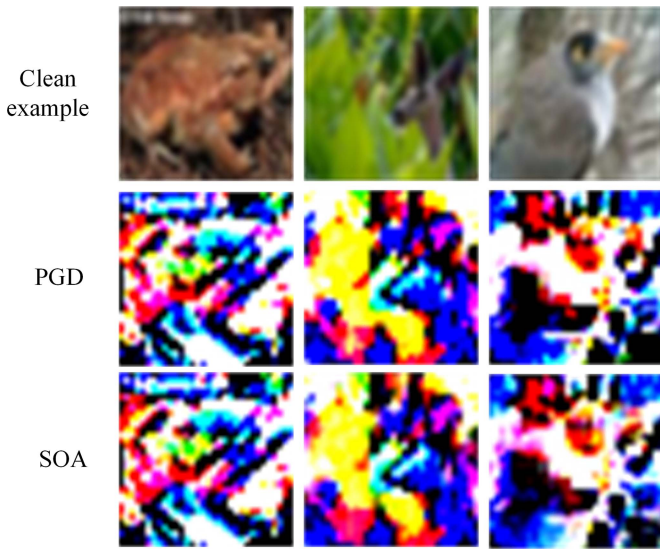


Fig. 2. Illustrative examples of pixel perturbations generated by PGD and SOA.

methods. This is because these second-order attack methods use the same sign function [15] as first-order methods to generate pixel perturbations. If sign function is directly applied to the second-order direction vector to generate pixel perturbations, the perturbations are almost the same as those of the first-order methods, which will not show the advantages of second-order optimization methods in terms of convergence. Fig. 2 illustrates the perturbations of PGD [9] and SOA [42]. From the figure, it is clear to see the perturbations of these two methods are of high similarity. Therefore, designing a proper perturbation generation strategy is the key to the success of second-order attack methods.

In this article, we propose a second-order optimization-based adaptive attack (SOAA) method, and at its core, an adaptive perturbation generation strategy is designed. We evaluate the performance of SOAA on multiple datasets and various models. The experimental results show SOAA is able to generate adversarial examples that cause misclassification of DCNNs in fewer iterations than the existing second-order attack methods and PGD, and has competitive time complexity.

The major contributions of this article are two-folds as follows.

- We propose an SOAA method, and at its core, an adaptive perturbation generation strategy is designed.
- Experimental results on multiple datasets and various models indicate that, the proposed SOAA with the designed adaptive perturbation generation strategy can successfully attack DCNNs within fewer iterations than the existing second-order attack methods and PGD, and exhibit competitive time complexity.

II. RELATED WORKS

In this section, we briefly review the literature related to first-order adversarial attack methods, second-order adversarial attack methods, and adversarial training (AT) methods.

A. First-Order Adversarial Attack

Adversarial examples are a crucial concept in the domain of deep learning robustness. An adversarial example is a carefully crafted sample intended to deceive deep learning models. These examples introduce subtle disturbances or perturbations to clean samples, which can lead the deep learning model to produce incorrect classification results. Interestingly, these minor perturbations are often imperceptible to the human eye [16], challenging our understanding and trust in deep learning models.

The two classic first-order adversarial attack methods are FGSM [8] and PGD [9]. FGSM first calculates the gradient of the model's loss function L concerning input x . It then uses the sign of this gradient multiplied by a small perturbation factor ε . This result is then added to the original image to produce the adversarial example x' , represented as

$$x' = x + \varepsilon \cdot \text{sign}(\nabla_x L(x, y)). \quad (1)$$

PGD is an iterative adversarial attack method. Due to its notable attacking capabilities and broad applicability, PGD is often used as a benchmark method for generating adversarial examples. It can be understood as an iterative version of FGSM, using smaller perturbations and including a projection operation $\text{clip}_{x,\varepsilon}$ in every step to ensure that the adversarial example always lies within a predefined perturbation range, represented as

$$x'_{i+1} = \text{clip}_{x,\varepsilon} \{x'_i + \alpha \cdot \text{sign}(\nabla_{x'_i} L(x'_i, y))\} \quad (2)$$

where i is the step number in PGD, and $\text{clip}(\cdot)$ is the clipping function that ensures the generated adversarial examples satisfy both the maximum perturbation limit ε constraint and the image's pixel value range.

In addition, several other first-order attack methods have also been proposed. Carlini and Wagner's attack [18] and DeepFool [19] attack computes the smallest perturbation needed to cause misclassification based on the first-order linear approximation of model's loss function. momentum iterative method (MIM) [17] and AutoAttack [20] are built on the foundation of PGD. MIM incorporates a momentum term, and AutoAttack is with an adaptive step size.

Although FGSM and PGD can effectively and efficiently attack deep learning models, one open question that still exists in this area is whether the second-order adversarial attack methods can obtain better performance compared to first-order attack methods.

B. Second-Order Adversarial Attack

Second-order optimization methods emphasize utilizing second-order derivative information to enhance the efficiency and accuracy of the optimization process. By integrating gradient and curvature information of the objective function, they can more precisely capture the function's geometric structure. Classic second-order optimization techniques include the Newton's method and natural gradient method, among others. Levenberg[21] and Marquardt[22] independently introduced the renowned Levenberg–Marquardt algorithm. This algorithm accelerates convergence by combining gradient descent and the Gauss–Newton method.

However, computing the complete second-order gradient information, especially in deep learning models, is often time-consuming. To address this issue, Broyden et al. [23] proposed the BFGS algorithm, a popular second-order optimization method that circumvents direct computation of the Hessian matrix. In response to the memory issues that BFGS encounters when dealing with large-scale problems, Nocedal et al. [24] introduced the L-BFGS optimization method. In the context of deep learning applications, researchers like Martens not only introduced the concept of second-order optimization but also demonstrated its practical impact within deep neural networks [25]. They further proposed the Kronecker-factored approximate curvature (KFAC) method [26]. This method accelerates optimization by leveraging the Fisher information matrix of neural networks and employs a Kronecker product decomposition strategy to approximate the Fisher information matrix, significantly reducing computational and storage overheads.

Recently, the AdaHessian optimizer [14] has garnered attention as an efficient second-order optimization method. By adaptively estimating the curvature information for each parameter, it greatly enhances the stability and speed of the training process. AdaHessian overcomes the computational challenges of traditional second-order optimization methods, making it feasible in large-scale deep learning models.

Based on the advancements in second-order optimization, there are a few studies on second-order attack methods. Yao et al. [41] introduced the TR-attack, which utilizes trust region optimization techniques. By approximating the loss function to the second order, this method calculates the optimal adversarial examples and adaptively adjusts the size of perturbations, enabling the rapid generation of effective adversarial examples. Li et al. [42] explored the use of second-order optimization techniques to enhance the efficiency of adversarial attacks. They use the first dominant eigenvector of Hessian matrix to approximate the second-order direction vector, which can generate adversarial examples fast. Qian et al. [43] proposed a second-order attack method, SOAE, which approximates the Hessian-inverse through optimization in the Krylov subspace, avoiding the computation of the full Hessian matrix and enhancing the efficiency of the attack. Specifically, the second-order direction vector is approximated by a linear combination of Hessian-vector product in the Krylov subspace.

Note that, the existing second-order attack methods directly apply sign function on the second-order direction vector to obtain the pixel perturbations, which limits their performance (detailed discussions are in Section III-B). In this article, we propose an SOAA method with an adaptive perturbation generation strategy.

C. AT Methods

To evaluate the performance of the attack methods, it is usually required to enhance the robustness of target models. Among various robustness enhancing techniques, AT has received widespread attention. In this section, we make an overview to the AT methods. Madry et al. [9] first introduced AT as an effective defensive mechanism. The fundamental idea

behind it is that during training, not only are the original clean samples used, but adversarial examples generated through adversarial attacks are also incorporated. This enables the model to gradually adapt to these complex and adversarial samples throughout its learning process, thus improving its robustness. Similar to Madry's AT, Zhang et al. [27] proposed the TRADES method, which also uses adversarial samples to bolster model robustness. The distinction lies in that, TRADES can concurrently optimize both the cross-entropy loss on clean samples and the KL divergence loss on adversarial samples during training. This dual-optimization approach helps the model maintain high performance when facing various types of attacks. Sehwal et al. [28], Goyal et al. [29], and others have achieved the objectives of AT by employing generative models to augment the training dataset. Such methods typically employ generative models, such as the denoising diffusion probabilistic models [30] to generate a larger set of training samples. These samples not only offer diversity but also exhibit high realism. By building on this, AT can more comprehensively cover the potential adversarial perturbation space, thereby enhancing the model's robustness.

III. SOAA METHOD

The optimization problem for adversarial attack is to find an adversarial example x' , which can maximize the loss function value of the target model, while being restricted within a predefined small neighborhood of its clean counterpart x , expressed in

$$\max_{x'} L(f(x'), y) \quad \text{s.t.} \quad \|x' - x\|_{\infty} \leq \varepsilon \quad (3)$$

where x is a clean example, x' is its adversarial counterpart, L is the loss function, f represents the target model, and $\|\cdot\|_{\infty}$ denotes the infinity norm, i.e., the maximum absolute value of the components.

In this section, we introduce an SOAA method to find adversarial examples. Generally, SOAA includes two steps. First in Section III-A, the second-order direction vector of SOAA is calculated based on AdaHessian [14]. Then in Section III-B, an adaptive perturbation generation strategy is designed to turn the second-order direction vector of Section III-A into pixel perturbations to generate the adversarial examples. The workflow of SOAA is shown in Fig. 3.

A. Calculation of Second-Order Direction Vector

The conventional second-order optimization method, namely the Newton's method [31], prominently features the computing of the inverse of Hessian matrix. Given a twice-differentiable function $f(x)$, the iterative formula of Newton's method is

$$x_{k+1} = x_k - H(x_k)^{-1} \cdot g(x_k) \quad (4)$$

where x_k is the solution at the k th iteration, $H(x_k)^{-1}$ is the inverse of the Hessian matrix at x_k , and $g(x_k)$ is the gradient of the function at x_k . The core idea of this formula is to utilize the second-order information of the function to update the current estimate, thereby converging to the function's minimum more quickly.

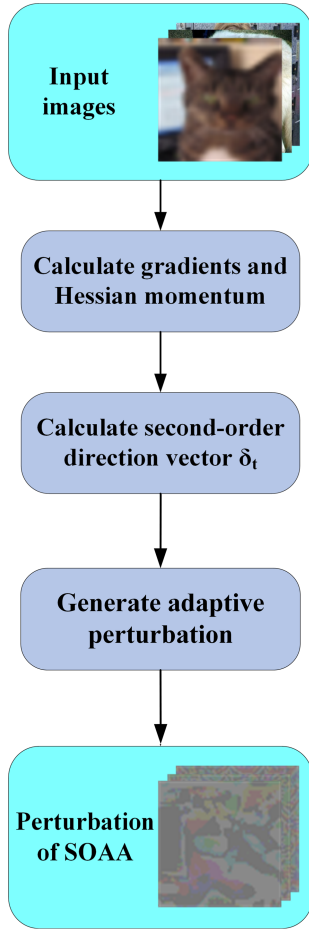


Fig. 3. Workflow of SOAA.

However, computing the inverse of Hessian matrix in each iteration is computationally expensive. The diagonal Hessian matrix is often used in place of the original Hessian matrix, which can be efficiently computed by using Hutchinson's method [32]. The following two equations express Hutchinson's method [33]:

$$\frac{\partial g z}{\partial x'} = \frac{\partial g}{\partial x'} z + g \frac{\partial z}{\partial x'} = \frac{\partial g}{\partial x'} z = H z \quad (5)$$

$$D = \text{diag}(H) = E[z \odot (H z)]. \quad (6)$$

Equation (5) computes the Hessian matrix times a vector z without having to explicitly form the Hessian matrix by the chain rule and z is independent of x . Equation (6) computes the diagonal Hessian matrix D using the expectation of $z \odot (H z)$, where z is a random vector following the Rademacher distribution.

The Hessian momentum during the update can be computed using the following equation [14]:

$$\bar{D}_t = \sqrt{\frac{(1 - \beta_2) \sum_{i=1}^t \beta_2^{t-i} D_i D_i}{1 - \beta_2^t}} \quad (7)$$

where $0 < \beta_2 < 1$ is the second moment hyperparameter, t is iteration number, D_i and is the diagonal Hessian matrix at iteration i . The Hessian momentum is used to smooth the local variations of the Hessian diagonal. Furthermore, the first

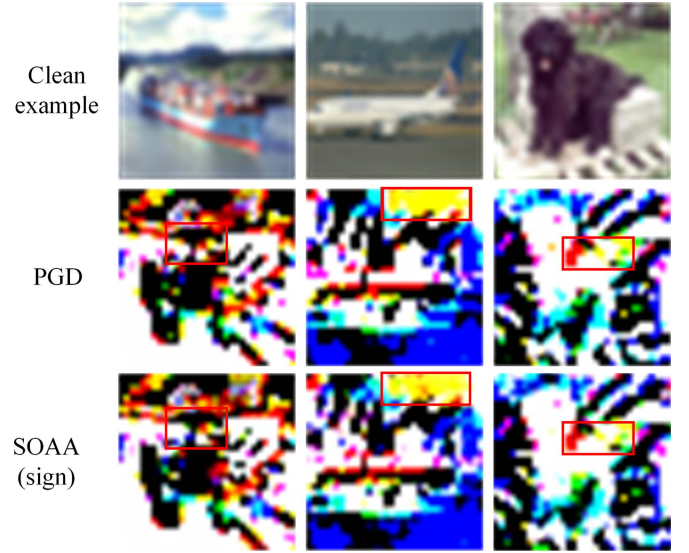


Fig. 4. Pixel perturbations generated by PGD and SOAA (sign).

moment m_t and the second moment v_t in each iteration can be calculated using (8) and (9) [14]

$$m_t = \frac{(1 - \beta_1) \sum_{i=1}^t \beta_1^{t-i} g_i}{1 - \beta_1^t} \quad (8)$$

$$v_t = (\bar{D}_t)^k = \left(\sqrt{\frac{(1 - \beta_2) \sum_{i=1}^t \beta_2^{t-i} D_i D_i}{1 - \beta_2^t}} \right)^k \quad (9)$$

where $0 < \beta_1, \beta_2 < 1$ are the hyperparameters for the first and second moments, respectively, k represents the Hessian weight, and g_i represents the gradient at iteration i .

The second-order direction vector δ_t in each iteration can be represented as follows:

$$\delta_t = m_t / v_t. \quad (10)$$

B. Adaptive Perturbation Generation Strategy

Motivation: Given a vector $a = \{a^1, a^2, \dots, a^N\} \in \mathbf{R}^N$, when applying sign function to a , we can obtain $p = \text{sign}(a) = \{p^1, p^2, \dots, p^N\}$, where p^i can be computed as follows:

$$p^i = \begin{cases} 1 & \text{if } a^i > 0 \\ 0 & \text{if } a^i = 0 \\ -1 & \text{if } a^i < 0. \end{cases} \quad (11)$$

It is straightforward to see that, directly applying sign function to the gradient and the second-order direction vector will result in the similar pixel perturbations according to the definition of sign function of (11).

Fig. 4 qualitatively compares the perturbations generated by PGD and SOAA of sign function. It can be observed that the two types of pixel perturbations are almost the same. Fig. 5 qualitatively compares the perturbations generated by PGD and SOAA of the adaptive perturbation strategy. It can be seen the difference between the two types of pixel perturbations is more

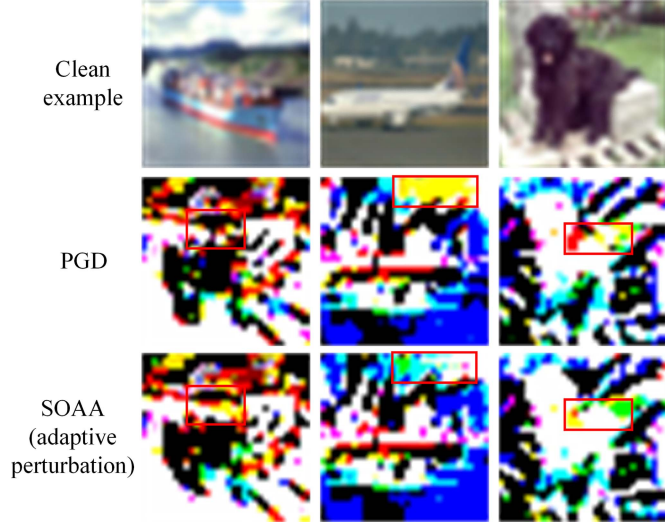


Fig. 5. Pixel perturbations generated by PGD and SOAA (adaptive perturbation).

TABLE I
RMSE $\uparrow$ OF PERTURBATIONS OF PGD AND SOAA (SIGN), AND OF PGD AND SOAA (ADAPTIVE PERTURBATION)

	CIFAR-10	SVNH	MNIST
SOAA(sign)	9.37×10^{-3}	1.16×10^{-2}	0.122
SOAA(adaptive perturbation)	1.11×10^{-2}	1.35×10^{-2}	0.169

TABLE II
COMPARISON OF THE CLASSIFICATION ACCURACY $\downarrow$ OF SOA, SOAE, AND SOAA WHEN REPLACING SIGN FUNCTION WITH THE ADAPTIVE PERTURBATION STRATEGY

Datasets	Attack methods	ResNet34		VGG19	
		Sign	Adaptive perturbation	Sign	Adaptive perturbation
CIFAR10	SOA [42]	29.04	28.91	43.46	43.33
	SOAE [43]	33.32	32.99	47.32	47.10
	SOAA	28.96	27.72	43.38	42.84
SHVN	SOA	31.09	30.89	54.44	54.16
	SOAE	34.01	33.88	58.42	57.89
	SOAA	31.04	29.07	54.51	53.79
MNIST	SOA	64.76	60.09	68.79	67.96
	SOAE	67.94	65.77	73.37	70.71
	SOAA	58.65	52.72	68.05	67.11

visually distinct than that in Fig. 4. For example, within the bounding box region, Fig. 5 shows a clear difference, while Fig. 4 demonstrates high similarity.

Table I quantitatively compares the perturbation differences between PGD and SOAA of sign function, and between PGD and SOAA of adaptive perturbation strategy in terms of RMSE. From the table, it is clear to see, on each dataset, the difference between PGD and SOAA of adaptive perturbation strategy is more significant than that between PGD and SOAA of sign function.

Table II demonstrates if replacing sign function of the existing second-order attack methods with adaptive perturbation strategy to generate pixel perturbations, the classification accuracy is lower, which means the attack success rate using adaptive perturbation strategy is higher than that using sign function.

Algorithm 1: SOAA.

Input: Target image x for attack, classifier f , true label y , iteration steps T , maximum perturbation limit ε , exponential decay rates β_1, β_2 , Hessian weight k .
Output: Adversarial example x' .
Initialize: $m_0 \leftarrow 0, v_0 \leftarrow 0, x'_0 \leftarrow x, \eta \leftarrow \varepsilon$;
1: **for** $t = 0$ to $T - 1$ **do**
2: $g_t \leftarrow \nabla_{x'_t} L(f(x'_t), y)$
3: $D_t \leftarrow \text{diag}(H_t)$
4: Update $\bar{D}_t$ based on Eq. (7)
5: Update m_t, v_t based on Eq. (8), Eq. (9)
6: $\delta_t \leftarrow m_t / v_t$
7: $\delta_t \leftarrow \frac{\lg(|\delta_t|) - \min(\lg(|\delta_t|))}{\max(\lg(|\delta_t|)) - \min(\lg(|\delta_t|))} \cdot \text{sign}(\delta_t)$
8: $x'_{t+1} = \text{clip}_{x, \varepsilon}(x'_t + \delta_t \cdot \eta)$
9: **if** Loss fluctuates and η was not adjusted in the previous iteration
10: $\eta \leftarrow \eta / 2$
11: **end if**
12: **end for**
13: $x' \leftarrow x'_T$
14: **return** x'

These all hint designing a novel perturbation generation strategy is of great importance for the superiority of the second-order attack methods. The details of the designed adaptive perturbation generation strategy are introduced as follows.

Adaptive perturbation generation strategy: In the designed adaptive perturbation generation strategy, we first compute the logarithm of the second-order direction vector at iteration t as follows:

$$\delta_t \leftarrow \frac{\lg(|\delta_t|) - \min(\lg(|\delta_t|))}{\max(\lg(|\delta_t|)) - \min(\lg(|\delta_t|))} \cdot \text{sign}(\delta_t) \quad (12)$$

$|\delta_t|$ means the absolute value of δ_t .

Then, based on the logarithm, the adversarial example at iteration $t + 1$ can be calculated as follows:

$$x'_{t+1} = \text{clip}_{x, \varepsilon}(x'_t + \delta_t \cdot \eta) \quad (13)$$

where η is a scaling coefficient, controlling the step size of perturbation, and is initially set to the perturbation limit ε .

Adaptively adjusting the step size can generate more delicate perturbations, thus enhancing the attack success rate [20]. If the loss value fluctuates at iteration t , which means the value increases fewer than $n\rho$ times in the last n consecutive iterations, and at iteration $t - 1$, η is not adjusted, we set $\eta = \eta / 2$ to make fine-grained search. In the experiments, n is set to 5, and ρ is set to 0.6. Section IV-E compares the attack performance of adaptive and nonadaptive perturbation strategies, and also that of different η adjusting strategies.

The algorithm of SOAA is depicted in Algorithm 1.

IV. EXPERIMENTS AND ANALYSIS

In this section, we evaluate SOAA performance on multiple datasets and models. First, we compare SOAA with PGD and three existing second-order attack methods [41], [42], [43] under

different iteration numbers and perturbation limits on VGG19 model and ResNet34 model. Second, the attack performance and time complexity of SOAA are examined on various models and under different defense mechanisms. Third, we perform the ablation studies with respect to perturbation generation strategy and adaptive settings. All experiments are carried out on a Xeon CPU at 4.0 GHz, equipped with 16 GB RAM and an RTX4080 16 GB GPU, using the PyTorch framework.

A. Experimental Setups

Baseline methods: Baseline methods used for comparison include PGD [9], SOA [42], SOAE [43], and TR [41] methods. PGD is a first-order attack method. SOA, SOAE, and TR are the second-order attack methods. SOA approximates the second-order direction vector using first dominant eigenvector of Hessian matrix, then employs sign function to generate pixel perturbations. SOAE approximates the second-order direction vector by a linear combination of the Hessian-vector product in the Krylov subspace, then uses sign function to generate pixel perturbations. TR utilizing trust region optimization techniques to calculate adversarial examples, is a second-order targeted attack method.

Evaluation metrics: We use classification accuracy (denoted as clacc), attack success rate (denoted as attsc), and time complexity to measure the performance of SOAA.

Clacc is defined as $\text{clacc} = \frac{\text{number of true predictions}}{\text{total number of samples}}$. It is straightforward to see that, the lower clacc is, the better performance an attack method has.

Attsc is defined as $\text{attsc} = \frac{\text{number of false predictions}}{\text{total number of samples}} = 1 - \text{clacc}$. It is straightforward to see that, the higher attsc is, the better performance an attack method has.

Time complexity is defined as the time cost of generating the adversarial examples by an attack method.

Parameter settings: For SOAA parameter settings, the exponential decay rates β_1 and β_2 are set to 0.9 and 0.999, respectively. The Hessian weight k is set to 1.

Datasets: In this study, the following three datasets are used.

CIFAR-10 [34]. This dataset consists of 60 000 color images of 32×32 pixels spanning across 10 categories, with each category containing 6000 images. The dataset is divided into five training batches and one test batch, with each batch containing 10 000 images.

MNIST [35]. This dataset features handwritten digits and comprises 60 000 training images and 10 000 test images. Each image is a 28×28 grayscale picture representing digits from 0 to 9.

SVHN [36]. Originating from Google Street View images, this dataset is meant for house number recognition. It includes over 600 000 colored digit images.

B. Attack Performance Comparison Between SOAA and Other Baselines Under Different Iteration Numbers and Perturbation Limits

We compare the attack success rate of SOAA with PGD [9], SOA [42], and SOAE [43] under different iteration numbers and perturbation limits using VGG19 model [37] and ResNet34

TABLE III
COMPARISON OF CLASSIFICATION ACCURACY $\downarrow$ UNDER SOAA, PGD, SOA, AND SOAE WITH DIFFERENT ITERATION NUMBERS

VGG19						
Attack methods	(a) CIFAR-10 (clean accuracy=75.69%)					
	$T=5$	$T=10$	$T=20$	$T=30$	$T=50$	$T=100$
PGD [9]	53.72	45.75	43.73	43.17	43.03	42.98
SOA [42]	44.52	43.73	43.46	43.22	43.16	43.02
SOAE [43]	53.39	51.57	47.32	46.98	46.67	46.55
SOAA	43.41	43.31	42.98	42.91	42.86	42.83
	(b) SVHN (clean accuracy=89.58%)					
	$T=5$	$T=10$	$T=20$	$T=30$	$T=50$	$T=100$
PGD	58.39	55.57	54.49	54.19	54.02	53.87
SOA	56.22	55.57	54.44	54.14	53.97	53.92
SOAE	59.01	58.77	58.42	58.11	57.93	57.66
SOAA	55.43	54.45	53.79	53.67	53.62	53.57
	(c) MNIST (clean accuracy=96.31%)					
	$T=5$	$T=10$	$T=20$	$T=30$	$T=50$	$T=100$
PGD	81.79	71.82	68.36	67.49	66.91	66.45
SOA	71.31	68.89	68.79	68.17	67.50	67.63
SOAE	76.36	72.27	73.37	72.88	72.67	72.25
SOAA	69.58	67.74	67.11	66.74	66.61	66.27
ResNet34						
	(a) CIFAR-10 (clean accuracy=72.14%)					
	$T=5$	$T=10$	$T=20$	$T=30$	$T=50$	$T=100$
PGD	36.89	30.01	29.13	28.93	28.77	28.63
SOA	31.27	29.81	29.04	28.90	28.76	28.65
SOAE	42.39	34.57	33.32	32.98	32.67	32.55
SOAA	29.25	28.44	27.74	27.30	26.51	26.18
	(b) SVHN (clean accuracy=86.52%)					
	$T=5$	$T=10$	$T=20$	$T=30$	$T=50$	$T=100$
PGD	39.35	33.23	31.08	30.35	29.83	29.45
SOA	35.04	32.04	31.09	30.30	29.77	29.36
SOAE	43.31	36.22	34.01	33.22	32.11	31.75
SOAA	33.66	31.03	29.07	27.80	27.10	26.93
	(c) MNIST (clean accuracy=97.52%)					
	$T=5$	$T=10$	$T=20$	$T=30$	$T=50$	$T=100$
PGD	81.35	72.30	64.89	62.56	60.83	59.58
SOA	70.38	67.58	64.76	62.69	61.25	61.03
SOAE	77.28	71.87	67.94	65.86	64.97	64.18
SOAA	66.60	57.90	52.72	49.02	46.47	45.53

model [39]. In Table III, the iteration number T is set to 5, 10, 20, 30, 50, and 100 for all three datasets, respectively. And the perturbation limits ε are 8/255 for CIFAR-10 and SVHN datasets, and 0.3 for MNIST dataset. PGD step size is 2/255 for CIFAR-10 and SVHN datasets, and 0.05 for MNIST dataset. From the table, it is clear to see the SOAA attack success rate is higher than PGD, SOA, and SOAE in each iteration number. In Table IV, the perturbation limits ε are set to 4/255, 6/255, 8/255, 12/255, 16/255, and 20/255 for CIFAR-10 and SVHN datasets, respectively, and 20/255, 40/255, 60/255, 0.3, 0.35, 0.4 for MNIST dataset, respectively. And the iteration number T is set to 20 for all three datasets. From the table, it is clear to see the SOAA attack success rate is higher than PGD, SOA, and SOAE under each perturbation limit. Both Tables III and IV demonstrate the superiority of the designed adaptive perturbation generation strategy over sign function, and indicate SOAA in general can find the adversarial examples that cause the misclassifications of DCNNs in fewer iterations than PGD, SOA, and SOAE.

Table V depicts the number of iterations required to perform a successful attack under SOAA, PGD, SOA, and SOAE on six illustrative examples. It can be seen that, on all six examples,

TABLE IV

COMPARISON OF CLASSIFICATION ACCURACY $\downarrow$ UNDER SOAA, PGD, SOA, AND SOAE WITH DIFFERENT PERTURBATION LIMITS

VGG19						
(a) CIFAR-10						
Attack methods	$\epsilon=4/255$	$\epsilon=6/255$	$\epsilon=8/255$	$\epsilon=12/255$	$\epsilon=16/255$	$\epsilon=20/255$
PGD [9]	60.73	51.82	43.73	27.79	16.59	9.91
SOA [42]	60.71	51.80	43.22	27.68	16.21	8.51
SOAE [43]	67.36	55.27	47.32	31.88	22.67	18.32
SOAA	60.36	51.73	43.13	27.18	15.01	7.30
(b) SVHN						
Attack methods	$\epsilon=4/255$	$\epsilon=6/255$	$\epsilon=8/255$	$\epsilon=12/255$	$\epsilon=16/255$	$\epsilon=20/255$
PGD	74.76	65.04	54.43	35.86	22.64	14.76
SOA	74.62	64.95	54.44	35.78	22.57	14.71
SOAE	79.31	68.21	58.42	38.65	26.35	18.69
SOAA	74.30	64.60	53.67	32.38	17.43	8.12
(c) MNIST						
Attack methods	$\epsilon=0.08$	$\epsilon=0.16$	$\epsilon=0.24$	$\epsilon=0.3$	$\epsilon=0.35$	$\epsilon=0.4$
PGD	93.66	88.88	80.25	68.36	34.41	4.29
SOA	93.66	88.85	79.80	68.79	27.79	2.74
SOAE	94.02	91.01	82.10	73.37	38.07	21.13
SOAA	93.53	88.32	79.11	67.11	5.56	0.07

ResNet34						
(a) CIFAR-10						
Attack methods	$\epsilon=4/255$	$\epsilon=6/255$	$\epsilon=8/255$	$\epsilon=12/255$	$\epsilon=16/255$	$\epsilon=20/255$
PGD	44.18	36.16	29.13	17.38	10.17	6.66
SOA	44.15	36.09	29.04	17.26	10.11	6.61
SOAE	47.36	39.80	33.32	20.78	15.27	12.37
SOAA	43.13	35.15	27.74	16.33	9.88	6.16
(b) SVHN						
Attack methods	$\epsilon=4/255$	$\epsilon=6/255$	$\epsilon=8/255$	$\epsilon=12/255$	$\epsilon=16/255$	$\epsilon=20/255$
PGD	49.93	40.10	31.08	18.75	12.38	9.37
SOA	49.90	40.01	31.09	18.77	12.41	9.19
SOAE	53.61	42.22	34.01	22.44	18.34	15.11
SOAA	48.81	38.32	29.14	16.30	9.70	6.06
(c) MNIST						
Attack methods	$\epsilon=0.08$	$\epsilon=0.16$	$\epsilon=0.24$	$\epsilon=0.3$	$\epsilon=0.35$	$\epsilon=0.4$
PGD	88.41	82.97	73.99	64.89	55.31	48.30
SOA	88.24	82.69	73.94	64.76	48.28	37.37
SOAE	91.32	85.11	76.39	67.94	58.08	52.10
SOAA	82.37	75.07	65.57	52.72	38.59	23.83

TABLE V

COMPARISON OF NUMBER OF ITERATIONS REQUIRED $\downarrow$ FOR SUCCESSFUL ATTACK UNDER SOAA, PGD, SOA, AND SOAE

						
PGD [9]	12	11	25	24	28	30
SOA [42]	16	12	24	21	19	26
SOAE [43]	48	33	127	66	109	61
SOAA	5	5	12	10	10	11

SOAA needs fewest iterations to generate adversarial counterparts to make the model misclassify. For example, on “cat” example of CIFAR-10 dataset, SOAA only needs 5 iterations to fool the model, while PGD, SOA, and SOAE require 12, 16, and 48 iterations, respectively. On “5” example of SVHN dataset, SOAA only needs 12 iterations to fool the model, while PGD, SOA, and SOAE require 25, 24, and 127 iterations, respectively. On “6” example of MNIST dataset, SOAA only needs ten iterations to perform a successful attack, while PGD, SOA, and SOAE require 28, 19, and 109 iterations, respectively.

Table VI compares the attack performance of SOAA with TR [41] using VGG19 model. Due to TR is a second-order

TABLE VI

COMPARISON OF CLASSIFICATION ACCURACY $\downarrow$ UNDER SOAA AND TR WITH DIFFERENT ITERATIONS IN TARGETED ATTACK SETTINGS

(a) CIFAR-10						
Attack methods	$T=5$	$T=10$	$T=20$	$T=30$	$T=50$	$T=100$
TR [41]	42.98	42.91	42.63	42.06	41.63	41.05
SOAA	42.46	42.07	41.47	41.21	41.11	40.94
(b) SVHN						
Attack methods	$T=5$	$T=10$	$T=20$	$T=30$	$T=50$	$T=100$
TR	53.53	53.42	52.87	52.15	51.33	50.60
SOAA	53.09	52.67	52.19	51.69	51.12	50.44
(c) MNIST						
Attack methods	$T=5$	$T=10$	$T=20$	$T=30$	$T=50$	$T=100$
TR	78.07	77.79	76.78	76.17	74.87	74.14
SOAA	70.83	68.38	66.49	66.36	66.18	65.93

TABLE VII

PSNR AND SSIM BETWEEN ORIGINAL IMAGES AND ATTACKED IMAGES BY SOAA

Labels	car	dog	frog	horse	truck
Original images					
Attacked images					
PSNR [46]	30.4871	30.7817	30.4943	30.4399	30.5930
SSIM [47]	0.9126	0.9611	0.9614	0.9410	0.9478

TABLE VIII

COMPARISON OF TIME COMPLEXITY UNDER SOAA, PGD, SOA, AND SOAE ON CIFAR-10 DATASET

Attack methods	VGG16	ResNet 34	VGG19	Dense Net121	WRN 34-10
Time cost $\downarrow$					
PGD [9]	16 s	10 s	20 s	24 s	1 min 25 s
SOA [42]	16 s	10 s	20 s	24 s	1 min 26 s
SOAE [43]	3 min 27 s	12 min 23 s	4 min 5 s	36 min 32 s	77 min 03 s
SOAA	42 s	34 s	48 s	1 min 47 s	5 min 58 s
Classification accuracy $\downarrow$					
PGD	24.84	16.51	27.79	16.78	36.68
SOA	24.70	16.69	27.64	16.71	36.61
SOAE	36.17	22.44	38.65	22.11	40.03
SOAA	23.98	15.87	27.18	16.38	35.64

targeted attack method, SOAA is set accordingly. From the table, it is clear to see, under each iteration number, the classification accuracy under SOAA is lower than TR, which indicates SOAA can find the adversarial examples fooling the model within fewer iterations than TR.

Table VII demonstrates the similarity between the original images and the SOAA-attacked counterparts in terms of PSNR [46] and SSIM [47]. The attacked images are of high similarity with the original images, and can make the VGG-19 model misclassify. For example, “car” image is incorrectly predicted to “truck,” “frog” image is incorrectly predicted to “ship,” and “horse” image is incorrectly predicted to “deer.”

TABLE IX
COMPARISON OF CLASSIFICATION ACCURACY ↓ UNDER SOAA AND PGD
WHEN FIXING THE TIME FOR GENERATING ADVERSARIAL EXAMPLES

VGG19			
(a) CIFAR-10			
Attack methods	30 s	1 min	2 min
PGD [9]	43.15	43.06	42.99
SOAA	43.12	43.01	42.93
(b) SVHN			
	1 min 30 s	3 min	5 min
PGD	54.19	53.98	53.73
SOAA	54.19	53.92	53.70
(c) MNIST			
	30 s	1 min	2 min
PGD	67.45	66.97	66.68
SOAA	66.91	66.23	66.04
ResNet34			
(a) CIFAR-10			
	30 s	1 min	2 min
PGD	28.76	28.56	27.94
SOAA	27.53	26.64	26.20
(b) SVHN			
	1 min 30 s	3 min	5 min
PGD	29.51	29.26	28.97
SOAA	29.07	27.43	26.91
(c) MNIST			
	30 s	1 min	2 min
PGD	60.83	59.44	58.63
SOAA	52.52	47.84	45.57

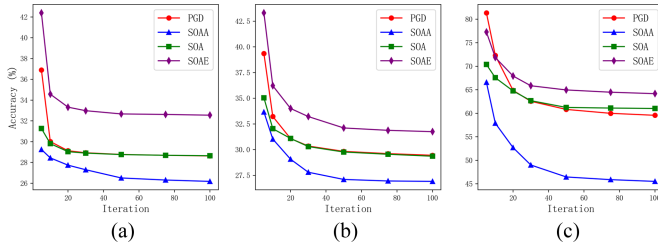


Fig. 6. Comparison of convergence under SOAA, PGD, SOA, and SOAE on ResNet34 model. (a) CIFAR-10. (b) SVHN. (c) MNIST.

C. Time Complexity Comparison Between SOAA and Other Baselines

In Table VIII, we compare the time complexity of SOAA, PGD, SOA, and SOAE on VGG16, VGG19, ResNet34, DenseNet121 [38], and WRN34-10 [40], respectively. The iteration number is set to 20. The perturbation limit is set to 12/255. And the dataset is CIFAR-10. From the table, it is clear to see, the time complexity of SOAA is higher than PGD and SOA, and lower than SOAE. This is straightforward as SOAA calculates the inverse of the approximate Hessian matrix, leading to a higher time cost for generating adversarial examples per iteration compared to PGD.

In Table IX, we fix the time for generating adversarial examples and compare the attack success rates of SOAA and PGD on VGG19 and ResNet34 models. The results on three datasets show that SOAA can achieve higher attack success rates than PGD in all cases.

Fig. 6 shows the convergence curves of SOAA, PGD, SOA, and SOAE on ResNet34 model and three datasets. It can be seen from the figure that, with the number of iterations increasing, the limit of the attack success rate of SOAA is higher than that of

TABLE X
COMPARISON OF CLASSIFICATION ACCURACY ↓ UNDER SOAA, PGD, SOA,
AND SOAE AGAINST DIFFERENT DEFENSE MODELS ON CIFAR-10 DATASET

Defense methods	Model	PGD [9]	SOA [42]	SOAE [43]	SOAA
Madry [9]	WRN34-10	43.78	43.66	47.02	41.92
TRADES [27]	WRN34-10	55.32	55.18	57.36	54.81
Gowal [28]	WRN28-10	66.15	66.04	68.89	65.69
Shwag [29]	RN152	65.56	65.48	68.51	65.20
Comdefend [44]	ResNet34	53.34	53.06	59.54	44.24
DiffPure [45]	ResNet34	46.37	46.78	47.10	40.95

TABLE XI
COMPARISON OF CLASSIFICATION ACCURACY ↓ BETWEEN ADAPTIVE
PERTURBATION AND SIGN FUNCTION

Perturbation computing strategies	CIFAR-10	SVNH	MNIST
Sign function	27.74	35.86	9.57
Adaptive perturbation	26.98	32.29	4.59

TABLE XII
COMPARISON OF CLASSIFICATION ACCURACY ↓ BETWEEN ADAPTIVE
PERTURBATION AND NONADAPTIVE PERTURBATION

Perturbation computing strategies	CIFAR-10	SVNH	MNIST
Non-adaptive perturbation	27.31	32.68	6.87
Adaptive perturbation ($\eta = \eta/1.5$)	27.07	32.36	5.27
Adaptive perturbation ($\eta = \eta/2$)	26.98	32.29	4.59
Adaptive perturbation ($\eta = \eta/4$)	27.32	32.50	4.82

PGD, SOA, and SOAE, demonstrating the superiority of SOAA over other baselines.

D. Attack Performance Comparison Between SOAA and Other Baselines Against Different Defense Mechanisms

In Table X, we compare the attack performance of SOAA, PGD, SOA, and SOAE against Madry [9], TRADES [27], Gowal [28], Shwag [29], Comdefend [44], and DiffPure [45] defense mechanisms on CIFAR-10 dataset. The perturbation limit and iteration number are set to 12/255 and 20, respectively. From the table, it is obvious to see, the attack success rate of SOAA is higher than that of PGD, SOA, and SOAE under each defense mechanism.

E. Ablation Study

The designed adaptive perturbation generation strategy is the key to the performance of SOAA. Here, we make an ablation study by comparing the adaptive perturbation generation strategy with sign function, and nonadaptive perturbation strategy. The nonadaptive perturbation strategy refers to removing the adaptive part from the adaptive perturbation strategy, which means during the perturbation calculation, the value of η is not adjusted after it is initially determined. In addition, as shown in line 10 of Algorithm 1, a division parameter is used to perform fine-grained search of η , we also conduct an ablation study by comparing the different values of the division parameter. In Table XI, it is clear to see SOAA of adaptive perturbation generation strategy can achieve higher attack success rate than SOAA of sign function on three datasets. In Table XII, it is clear to see SOAA of adaptive perturbation strategy can achieve higher attack success rate than SOAA of nonadaptive strategy on three datasets. Furthermore, in adaptive perturbation strategy, when

the division parameter is set to 2 to perform fine-grained search for η , better attack performance can be achieved than values of 1.5 and 4.

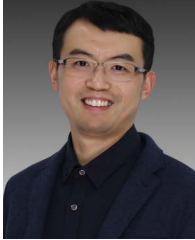
V. CONCLUSION

Adversarial attacks based on first-order optimization are widely used in the robustness analysis of DCNNs. Second-order optimization has a faster convergence rate than first-order optimization, but adversarial attacks based on second-order optimization have not yet been fully explored. In this article, we propose an SOAA method, and at its core, an adaptive perturbation generation strategy is designed. Experimental results on multiple datasets and various models show that the proposed SOAA can make a successful attack faster than the existing second-order attack methods and PGD, and has competitive time complexity. The work in this article demonstrates an effective and efficient adversarial attack method, which can be an alternative to the first-order methods for the robustness analysis of DCNNs. In future work, we will strive to reduce the time complexity of SOAA and study the AT on SOAA.

REFERENCES

- [1] S. W. Shieh, J. Voas, P. Laplante, and J. Rupe, "Reliability engineering in a time of rapidly converging technologies," *IEEE Trans. Rel.*, vol. 73, no. 1, pp. 73–82, Mar. 2024.
- [2] S. Al-Maliki et al., "Toward improved reliability of deep learning based systems through online relabeling of potential adversarial attacks," *IEEE Trans. Rel.*, vol. 72, no. 4, pp. 1367–1382, Dec. 2023.
- [3] Z. Chen, D. Zhou, E. Zio, T. Xia, and E. Pan, "A deep learning feature fusion based health index construction method for prognostics using multiobjective optimization," *IEEE Trans. Rel.*, vol. 72, no. 3, pp. 1038–1052, Sep. 2023.
- [4] C. Szegedy et al., "Intriguing properties of neural networks," in *Proc. Int. Conf. Learn. Representations*, 2014.
- [5] P. Qi, T. Jiang, L. Wang, X. Yuan, and Z. Li, "Detection tolerant black-box adversarial attack against automatic modulation classification with deep learning," *IEEE Trans. Rel.*, vol. 71, no. 2, pp. 674–686, Jun. 2022.
- [6] N. Carlini and D. Wagner, "Audio adversarial examples: Targeted attacks on speech-to-text," *Proc. IEEE Secur. Privacy Workshops*, San Francisco, CA, USA, 2018, pp. 1–7.
- [7] W. E. Zhang, Q. Z. Sheng, A. Alhazmi, and C. Li, "Adversarial attacks on deep-learning models in natural language processing: A survey," *ACM Trans. Intell. Syst. Technol.*, vol. 11, no. 3, pp. 1–41, 2020.
- [8] I. J. Goodfellow, J. Shlens, and C. Szegedy, "Explaining and harnessing adversarial examples," in *Proc. Int. Conf. Learn. Representations*, 2015.
- [9] A. Madry, A. Makelov, L. Schmidt, D. Tsipras, and A. Vladu, "Towards deep learning models resistant to adversarial attacks," in *Proc. Int. Conf. Learn. Representations*, 2018.
- [10] H. Zhang, Y. Avrithis, T. Furon, and L. Amsaleg, "Walking on the edge: Fast, low-distortion adversarial examples," *IEEE Trans. Inf. Forensics Secur.*, vol. 16, pp. 701–713, 2021.
- [11] A. Athalye, N. Carlini, and D. Wagner, "Obfuscated gradients give a false sense of security: Circumventing defenses to adversarial examples," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 274–283.
- [12] R. Shrestha, "Natural gradient methods: Perspectives, efficient-scalable approximations, and analysis," 2023, *arXiv:2303.05473*.
- [13] P. Xu, F. Roosta, and M. W. Mahoney, "Second-order optimization for non-convex machine learning: An empirical study," in *Proc. SIAM Int. Conf. Data Mining Soc. Ind. Appl. Math.*, 2020, pp. 199–207.
- [14] Z. Yao, A. Gholami, S. Shen, M. Mustafa, K. Keutzer, and M. Mahoney, "ADAHESSIAN: An adaptive second order optimizer for machine learning," in *Proc. AAAI Conf. Artif. Intell.*, 2021, vol. 35, no. 12, pp. 10665–10673.
- [15] M. Courbariaux, I. Hubara, D. Soudry, R. El-Yaniv, and Y. Bengio, "Binarized neural networks: Training deep neural networks with weights and activations constrained to 1 or -1," 2016, *arXiv:1602.02830*.
- [16] Y. Lin, H. Zhao, X. Ma, Y. Tu, and M. Wang, "Adversarial attacks in modulation recognition with convolutional neural networks," *IEEE Trans. Rel.*, vol. 70, no. 1, pp. 389–401, Mar. 2021.
- [17] Y. Dong et al., "Boosting adversarial attacks with momentum," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2018, Salt Lake City, UT, USA, 2018, pp. 9185–9193.
- [18] N. Carlini and D. Wagner, "Towards evaluating the robustness of neural networks," in *Proc. IEEE Symp. Secur. Privacy (S&P)*, 2017, San Jose, CA, USA, 2017, pp. 39–57.
- [19] S. M. Moosavi-Dezfooli, A. Fawzi, and P. Frossard, "DeepFool: A simple and accurate method to fool deep neural networks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 2574–2582.
- [20] F. Croce and M. Hein, "Reliable evaluation of adversarial robustness with an ensemble of diverse parameter-free attacks," in *Proc. Int. Conf. Mach. Learn.*, 2020, pp. 2206–2216.
- [21] K. Levenberg, "A method for the solution of certain non-linear problems in least squares," *Quart. Appl. Math.*, vol. 2, no. 2, pp. 164–168, 1944.
- [22] D. W. Marquardt, "An algorithm for least-squares estimation of nonlinear parameters," *J. Soc. Ind. Appl. Math.*, vol. 11, no. 2, pp. 431–441, 1963.
- [23] C. G. Broyden, "The convergence of a class of double-rank minimization algorithms 1. general considerations," *IMA J. Appl. Math.*, vol. 6, no. 1, pp. 76–90, 1970.
- [24] J. Nocedal, "Updating quasi-Newton matrices with limited storage," *Math. Computation*, vol. 35, no. 151, pp. 773–782, 1980.
- [25] J. Martens, "Deep learning via Hessian-free optimization," in *Proc. Int. Conf. Mach. Learn.*, 2010, pp. 735–742.
- [26] J. Martens and R. Grosse, "Optimizing neural networks with Kronecker-factored approximate curvature," in *Proc. Int. Conf. Mach. Learn.*, 2015, pp. 2408–2417.
- [27] H. Zhang, Y. Yu, J. Jiao, E. Xing, L. El Ghauoui, and M. Jordan, "Theoretically principled trade-off between robustness and accuracy," in *Proc. Int. Conf. Mach. Learn.*, 2019, pp. 7472–7482.
- [28] V. Schwag et al., "Robust learning meets generative models: Can proxy distributions improve adversarial robustness," in *Proc. Int. Conf. Learn. Representations*, 2022.
- [29] S. Goyal, C. Qin, J. Uesato, T. Mann, and P. Kohli, "Uncovering the limits of adversarial training against norm-bounded adversarial examples," 2020, *arXiv:2010.03593*.
- [30] Y. Zhang, X. Ding, G. Li, Z. Zhang, and K. Yang, "Offline real-world wireless interference signal classification algorithm utilizing denoising diffusion probability model," *IEEE Signal Process. Lett.*, vol. 30, pp. 1132–1136, 2023.
- [31] J. Rafati and R. F. Marica, "Quasi-newton optimization methods for deep learning applications," *Deep Learn. Appl. Adv. Intel. Syst. Comput.*, vol. 1098, pp. 9–38, 2020.
- [32] M. F. Hutchinson, "A stochastic estimator of the trace of the influence matrix for Laplacian smoothing splines," *Commun. Statist.-Simul. Computation*, vol. 18, no. 2, pp. 1059–1076, 1989.
- [33] H. Avron and S. Toledo, "Randomized algorithms for estimating the trace of an implicit symmetric positive semi-definite matrix," *J. ACM (JACM)*, vol. 58, no. 2, pp. 1–34, 2011.
- [34] A. Krizhevsky and G. Hinton, "Learning multiple layers of features from tiny images," Tech. Rep., 2009.
- [35] Y. LeCun, "The MNIST database of handwritten digits," 1998. [Online]. Available: <http://yann.lecun.com/exdb/mnist/>
- [36] Y. Netzer et al., "Reading digits in natural images with unsupervised feature learning," *NIPS Workshop Deep Learn. Unsupervised Feature Learn.*, vol. 2011, no. 5, 2011, Art. no. 4.
- [37] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," in *Proc. Int. Conf. Learn. Representations*, 2014.
- [38] G. Huang, Z. Liu, L. V. D. Maaten, and K. Q. Weinberger, "Densely connected convolutional networks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2017, pp. 2261–2269.
- [39] S. Zagoruyko and N. Komodakis, "Wide residual networks," in *Proc. Brit. Mach. Vis. Conf.*, 2016.
- [40] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 770–778.
- [41] Z. Yao, A. Gholami, P. Xu, K. Keutzer, and M. W. Mahoney, "Trust region based adversarial attack on neural networks," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 11342–11351.
- [42] B. Li, C. Chen, W. Wang, and L. Carin, "Second-order adversarial attack and certifiable robustness," 2018, *arXiv:1809.03113*.

- [43] Y. Qian, Y. Wang, B. Wang, Z. Gu, Y. Guo, and W. Swaileh, "Hessian-free second-order adversarial examples for adversarial learning," 2022, *arXiv:2207.01396*.
- [44] X. Jia, X. Wei, X. Cao, and H. Foroosh, "ComDefend: An efficient image compression model to defend adversarial examples," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 6077–6085.
- [45] W. Nie, B. Guo, Y. Huang, C. Xiao, A. Vahdat, and A. Anandkumar, "Diffusion models for adversarial purification," in *Proc. Int. Conf. Mach. Learn.*, 2022.
- [46] H. Quan and G. Ghanbari, "Scope of validity of PSNR in image/video quality assessment," *Electron. Lett.*, vol. 44, no. 13, pp. 800–801, 2008.
- [47] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, "Image quality assessment: From error visibility to structural similarity," *IEEE Trans. image Process.*, vol. 13, no. 4, pp. 600–612, Apr. 2004.



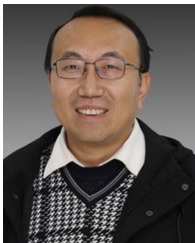
Jin Ding received the Ph.D. degree in control science and engineering from Zhejiang University, Hangzhou, China, in 2014.

He is currently an Associated Professor with the Zhejiang University of Science and Technology, Hangzhou, China. His research interests include image processing, 3-D vision, and vision SLAM.



Jie-Chao Zhao received the B.S. and M.S. degrees in automation from the Zhejiang University of Science and Technology, Hangzhou, China, in 2022 and 2025, respectively.

He is currently a Research Engineer in integer-AI with Information Technology (Hangzhou) Company Ltd., Hangzhou, China. His research interests include deep learning, adversarial robustness, and image processing.



Yong-Zhi Sun received the Ph.D. degree in electrical engineering from Zhejiang University, Hangzhou, China, in 2004.

He is currently a Research Faculty with the Zhejiang University of Science and Technology, Hangzhou, China. His research interests include embedded systems, intelligent vehicle, and robot force transducer.



Ping Tan (Member, IEEE) received the Ph.D. degree in control science and engineering from Zhejiang University, Hangzhou, China, in 2014.

He is currently a Professor with the Zhejiang University of Science and Technology, Hangzhou. His research interests include intelligent and safety control systems, rail transit signal systems, and high-speed railway monitoring systems.



Ji-En Ma (Member, IEEE) received the Ph.D. degree in mechanical and energy engineering from Zhejiang University, Hangzhou, China, in 2009.

She is currently a Professor with Zhejiang University. Her research interests include electrical machinery and appliances, applied economics, and high-speed railway.



You-Tong Fang received the Ph.D. degree in electrical engineering from the Hebei University of Technology, Tianjin, China, in 2001.

He is currently a Professor with Zhejiang University, Hangzhou, China. His research interests include permanent magnet motor, high-speed train traction power, wind turbine generator, and nuclear power.